PYTHON ACTIVITY-BASED LEARNING (ABL)

1. Examination Result and Grace-Mark Validation

Selected from the supplied list: Case study 11.

1. Problem Statement

Write a Python program to read marks in three subjects, check each subject's pass mark, and apply at most 5 grace marks to one failed subject if that is enough to make the student pass. Marks must not exceed 100.

2. Objectives

- Read and store subject marks.
- Check subject-wise pass criteria.
- Apply the stated grace-mark rule and print the result.

3. Assumptions

- There are three subjects.
- Pass mark is 35 in each subject.
- At most 5 grace marks can be added to one subject that has marks from 30 to 34.
- Input marks must be between 0 and 100.

4. Algorithm

1. Start.

2. Input three subject marks.

3. Validate that each mark is between 0 and 100.

4. If exactly one subject has marks from 30 to 34, add enough grace marks to reach 35, up to 5.

5. Check whether all final marks are at least 35.

6. Display final marks and pass/fail result.

7. Stop.

5. Flowchart (Text Form)

START → Input marks → Valid marks? → Apply eligible grace marks → All subjects ≥ 35? → Display result → STOP

6. Pseudocode

```
READ marks[3]\nIF any mark < 0 OR > 100: PRINT invalid input\nELSE:\n  Find subjects with marks 30–34\n  IF exactly one such subject and all other subjects ≥ 35:\n    Add required grace marks to that subject\n  IF every final mark ≥ 35: PRINT PASS\n  ELSE: PRINT FAIL
```

7. Python Source Code

```
marks = []
for i in range(3):
    mark = int(input(f"Enter marks for subject {i + 1}: "))
    if mark < 0 or mark > 100:
        print("Invalid marks")
        break
    marks.append(mark)

if len(marks) == 3:
    failed = [i for i, mark in enumerate(marks) if mark < 35]
```

```
if len(failed) == 1 and 30 <= marks[failed[0]] <= 34:
    other_passed = all(marks[i] >= 35 for i in range(3) if i != failed[0])
    if other_passed:
        marks[failed[0]] = 35
        print("Grace marks applied:  subject", failed[0] + 1)

print("Final marks:", marks)
if all(mark >= 35 for mark in marks):
    print("Result: PASS")
else:
    print("Result: FAIL")
```

8. Test Cases

Input / Scenario	Expected result
40, 32, 55	Grace applied to subject 2; final marks [40, 35, 55]; PASS
34, 28, 70	No eligible single-subject grace; FAIL
40, 50, 60	No grace needed; PASS

9. Sample Output

```
Enter marks for subject 1: 40
Enter marks for subject 2: 32
Enter marks for subject 3: 55
Grace marks applied: subject 2
Final marks: [40, 35, 55]
Result: PASS
```

Screenshot note: Run the program with the sample input above and insert a screenshot of the console output here if your faculty requires an actual image.

10. Conclusion

The program checks subject-wise pass criteria and applies the defined grace-mark rule without exceeding the maximum marks.

2. Attendance Recovery Computation

Selected from the supplied list: Case study 12.

1. Problem Statement

Calculate a student's current attendance percentage and determine the minimum number of consecutive classes the student must attend to reach a required attendance percentage.

2. Objectives

- Calculate attendance percentage.
- Determine how many additional classes must be attended.
- Explain the result using a loop.

3. Assumptions

- Attendance percentage = attended classes / conducted classes × 100.
- The student attends every future class.
- The required threshold is 75%.
- Conducted classes must be greater than zero.

4. Algorithm

1. Start.

2. Input attended and conducted classes.

3. Validate the values.

4. Calculate current percentage.

5. Increase both attended and conducted counts one class at a time until the percentage reaches 75%.

6. Display current percentage and additional classes needed.

7. Stop.

5. Flowchart (Text Form)

START → Input attended/conducted → Valid input? → Calculate percentage → Loop: add one attended class until threshold reached → Display → STOP

6. Pseudocode

```
READ attended, conducted\nIF conducted <= 0 OR attended < 0 OR attended > conducted: PRINT invalid\nELSE:\n
percentage = attended / conducted * 100\n  extra = 0\n  WHILE percentage < 75:\n    attended = attended + 1\n
conducted = conducted + 1\n    extra = extra + 1\n    percentage = attended / conducted * 100\n  PRINT percentage,
extra
```

7. Python Source Code

```
attended = int(input("Classes attended: "))
conducted = int(input("Total classes conducted: "))
required = 75

if conducted <= 0 or attended < 0 or attended > conducted:
    print("Invalid attendance data")
else:
    current = attended / conducted * 100
    extra = 0
    a, c = attended, conducted

    while (a / c) * 100 < required:
        a += 1
```

```
        c += 1
        extra += 1

    print(f"Current attendance: {current:.2f}%")
    print("Additional classes to attend:", extra)
    print(f"Attendance after attending them: {(a / c) * 100:.2f}%")
```

8. Test Cases

Input / Scenario	Expected result
60 attended, 100 conducted	Current attendance: 60.00%; additional classes: 60; final: 75.00%
80 attended, 100 conducted	Current attendance: 80.00%; additional classes: 0
0 attended, 0 conducted	Invalid attendance data

9. Sample Output

```
Classes attended: 60
Total classes conducted: 100
Current attendance: 60.00%
Additional classes to attend: 60
Attendance after attending them: 75.00%
```

Screenshot note: Run the program with the sample input above and insert a screenshot of the console output here if your faculty requires an actual image.

10. Conclusion

The program uses a loop to find the minimum number of future classes needed to reach the attendance requirement.

3. ATM Transaction Sequence Simulation

Selected from the supplied list: Case study 13.

1. Problem Statement

Simulate a sequence of deposits and withdrawals, maintain the account balance, reject invalid transactions, and show a transaction summary.

2. Objectives

- Process deposit and withdrawal transactions.
- Prevent withdrawals that exceed the balance.
- Count successful and rejected transactions.

3. Assumptions

- The opening balance is entered by the user and cannot be negative.
- Enter D for deposit, W for withdrawal, and X to finish.
- Transaction amounts must be positive.
- A withdrawal cannot exceed the current balance.

4. Algorithm

1. Start.

2. Input opening balance.

3. Repeatedly input transaction type and amount until X is entered.

4. Process valid deposits and affordable withdrawals.

5. Reject invalid transaction types, non-positive amounts, and unaffordable withdrawals.

6. Display final balance and transaction counts.

7. Stop.

5. Flowchart (Text Form)

START → Opening balance → Enter transaction → D/W/X? → Validate and update balance → Repeat or show summary → STOP

6. Pseudocode

```
READ balance\nSET successful = 0, rejected = 0\nREPEAT:\n  READ transaction type\n  IF type is X: EXIT LOOP\n
READ amount\n  IF amount <= 0: reject\n  ELSE IF type is D: balance += amount; successful += 1\n  ELSE IF type is
W AND amount <= balance: balance -= amount; successful += 1\n  ELSE: reject\nPRINT balance and counts
```

7. Python Source Code

```
balance = float(input("Opening balance: "))
successful = 0
rejected = 0

if balance < 0:
    print("Invalid opening balance")
else:
    while True:
        kind = input("Transaction (D=deposit, W=withdraw, X=exit): ").upper()
        if kind == "X":
            break
        if kind not in ("D", "W"):
            print("Rejected: invalid transaction type")
            rejected += 1
```

```
            continue

        amount = float(input("Amount: "))
        if amount <= 0:
            print("Rejected: amount must be positive")
            rejected += 1
        elif kind == "D":
            balance += amount
            successful += 1
            print("Deposit successful")
        elif amount <= balance:
            balance -= amount
            successful += 1
            print("Withdrawal successful")
        else:
            print("Rejected: insufficient balance")
            rejected += 1

    print(f"Final balance: Rs. {balance:.2f}")
    print("Successful transactions:", successful)
    print("Rejected transactions:", rejected)
```

8. Test Cases

Input / Scenario	Expected result
Balance 1000; D 500; W 200; X	Final balance Rs. 1300.00; successful 2; rejected 0
Balance 300; W 500; X	Withdrawal rejected; final balance Rs. 300.00
Balance 100; D -10; X	Deposit rejected; final balance Rs. 100.00

9. Sample Output

```
Opening balance: 1000
Transaction (D=deposit, W=withdraw, X=exit): D
Amount: 500
Deposit successful
Transaction (D=deposit, W=withdraw, X=exit): W
Amount: 200
Withdrawal successful
Transaction (D=deposit, W=withdraw, X=exit): X
Final balance: Rs. 1300.00
Successful transactions: 2
Rejected transactions: 0
```

Screenshot note: Run the program with the sample input above and insert a screenshot of the console output here if your faculty requires an actual image.

10. Conclusion

The program demonstrates transaction processing, loop control, input validation, and balance updates.

4. Warehouse Inventory Reconciliation

Selected from the supplied list: Case study 14.

1. Problem Statement

For multiple products, calculate expected closing stock from opening stock, incoming stock, and dispatched stock. Compare it with the actual closing stock and report discrepancies.

2. Objectives

- Calculate expected closing stock.
- Compare expected and actual stock.
- Identify discrepancies and replenishment needs.

3. Assumptions

- All quantities are whole numbers and non-negative.
- Expected closing stock = opening + incoming − dispatched.
- A negative expected quantity indicates inconsistent records.

4. Algorithm

1. Start.

2. Input the number of products.

3. For each product, input opening, incoming, dispatched, and actual closing quantities.

4. Calculate expected closing stock.

5. Compare expected with actual closing stock.

6. Print matched or discrepancy and shortage/surplus amount.

7. Stop.

5. Flowchart (Text Form)

START → Input product data → Calculate expected stock → Compare actual stock → Report discrepancy → Next product → STOP

6. Pseudocode

```
READ number_of_products\nFOR each product:\n  READ opening, incoming, dispatched, actual\n  expected = opening + incoming - dispatched\n  difference = actual - expected\n  IF difference == 0: PRINT matched\n  ELSE: PRINT discrepancy and difference
```

7. Python Source Code

```
n = int(input("Number of products: "))

for i in range(n):
    print(f"Product {i + 1}")
    opening = int(input("Opening stock: "))
    incoming = int(input("Incoming stock: "))
    dispatched = int(input("Dispatched stock: "))
    actual = int(input("Actual closing stock: "))

    if min(opening, incoming, dispatched, actual) < 0:
        print("Invalid quantity")
        continue

    expected = opening + incoming - dispatched
    difference = actual - expected
```

```
print("Expected closing stock:", expected)

if difference == 0:
    print("Stock matched")
elif difference < 0:
    print("Shortage:", abs(difference))
else:
    print("Surplus:", difference)
```

8. Test Cases

Input / Scenario	Expected result
Opening 50, incoming 20, dispatched 30, actual 40	Expected 40; Stock matched
Opening 50, incoming 10, dispatched 20, actual 35	Expected 40; Shortage 5
Opening 20, incoming 10, dispatched 5, actual 30	Expected 25; Surplus 5

9. Sample Output

```
Number of products: 1
Product 1
Opening stock: 50
Incoming stock: 20
Dispatched stock: 30
Actual closing stock: 35
Expected closing stock: 40
Shortage: 5
```

Screenshot note: Run the program with the sample input above and insert a screenshot of the console output here if your faculty requires an actual image.

10. Conclusion

The program reconciles stock records and reports whether the warehouse has matching stock, a shortage, or a surplus.

5. Student Marks Matrix Analysis

Selected from the supplied list: Case study 42.

1. Problem Statement

Store marks for multiple students and subjects in a nested list. Calculate each student's total and average, find the class average for each subject, and identify the top student.

2. Objectives

- Use a nested list to store marks.
- Calculate row-wise and column-wise statistics.
- Identify the student with the highest total.

3. Assumptions

- Each student has marks in three subjects.
- Marks are between 0 and 100.
- At least one student is entered.

4. Algorithm

1. Start.

2. Input student names and three marks for each student.

3. Store marks in a nested list.

4. Calculate each student's total and average.

5. Calculate each subject's class average.

6. Find the student with the highest total.

7. Display all results.

8. Stop.

5. Flowchart (Text Form)

START → Input names and marks → Store nested list → Calculate totals/averages → Find top student → Display → STOP

6. Pseudocode

```
FOR each student: READ name and 3 marks; store marks\nFOR each student row: calculate total and average\nFOR each subject column: calculate class average\nFind maximum student total\nPRINT summaries and top student
```

7. Python Source Code

```
names = []
marks = []
students = int(input("Number of students: "))

if students <= 0:
    print("Enter at least one student")
else:
    for i in range(students):
        name = input("Student name: ")
        row = []
        for subject in range(3):
            mark = int(input(f"Marks for subject {subject + 1}: "))
            row.append(mark)
        names.append(name)
        marks.append(row)
```

```
totals = [sum(row) for row in marks]
for i in range(students):
    print(names[i], "Total:", totals[i],
          "Average:", round(totals[i] / 3, 2))

for subject in range(3):
    average = sum(marks[i][subject] for i in range(students)) / students
    print(f"Subject {subject + 1} class average: {average:.2f}")

top = totals.index(max(totals))
print("Top student:", names[top])
```

8. Test Cases

Input / Scenario	Expected result
Asha: 80, 70, 90; Ravi: 60, 75, 65	Asha total 240 average 80; Ravi total 200 average 66.67; top student Asha
One student: 50, 50, 50	Total 150; average 50; top student is that student
0 students	Enter at least one student

9. Sample Output

```
Number of students: 2
Student name: Asha
Marks for subject 1: 80
Marks for subject 2: 70
Marks for subject 3: 90
Student name: Ravi
Marks for subject 1: 60
Marks for subject 2: 75
Marks for subject 3: 65
Asha Total: 240 Average: 80.0
Ravi Total: 200 Average: 66.67
Subject 1 class average: 70.00
Subject 2 class average: 72.50
Subject 3 class average: 77.50
Top student: Asha
```

Screenshot note: Run the program with the sample input above and insert a screenshot of the console output here if your faculty requires an actual image.

10. Conclusion

The program demonstrates nested lists and row-wise and column-wise analysis of student marks.

6. Tuple-Based Product Catalogue Audit

Selected from the supplied list: Case study 49.

1. Problem Statement

Store product ID, price, and stock quantity in tuples. Calculate each product's inventory value and identify products that need replenishment.

2. Objectives

- Represent product records using tuples.
- Calculate inventory value.
- Flag products with low stock.

3. Assumptions

- Each product record is a tuple: (product_id, price, quantity).
- Price and quantity cannot be negative.
- Replenishment is required when stock is below 5 units.

4. Algorithm

1. Start.

2. Input the number of products.

3. Read product ID, price, and quantity for each product.

4. Store each record as a tuple.

5. Calculate price multiplied by quantity.

6. Report products with quantity below 5.

7. Stop.

5. Flowchart (Text Form)

START → Input product records → Store tuples → Calculate inventory value → Stock < 5? → Report → STOP

6. Pseudocode

```
READ n\nFOR each product:\n  READ id, price, quantity\n  record = (id, price, quantity)\n  value = price *
quantity\n  PRINT record and value\n  IF quantity < 5: PRINT replenishment required
```

7. Python Source Code

```
products = []
n = int(input("Number of products: "))

for i in range(n):
    product_id = input("Product ID: ")
    price = float(input("Price: "))
    quantity = int(input("Stock quantity: "))

    if price < 0 or quantity < 0:
        print("Invalid product data")
        continue

    products.append((product_id, price, quantity))

for product in products:
    product_id, price, quantity = product
    value = price * quantity
    print("Product:", product_id,
```

```
        "Inventory value: Rs.", f"{value:.2f}")
    if quantity < 5:
        print("Replenishment required")
```

8. Test Cases

Input / Scenario	Expected result
P01, price 100, quantity 10	Inventory value Rs. 1000.00; no replenishment
P02, price 250, quantity 3	Inventory value Rs. 750.00; replenishment required
P03, price 50, quantity 0	Inventory value Rs. 0.00; replenishment required

9. Sample Output

```
Number of products: 2
Product ID: P01
Price: 100
Stock quantity: 10
Product ID: P02
Price: 250
Stock quantity: 3
Product: P01 Inventory value: Rs. 1000.00
Product: P02 Inventory value: Rs. 750.00
Replenishment required
```

Screenshot note: Run the program with the sample input above and insert a screenshot of the console output here if your faculty requires an actual image.

10. Conclusion

The program uses tuples and tuple unpacking to audit product records, calculate stock value, and flag low inventory.

7. Bus Passenger Occupancy Simulation

Selected from the supplied list: Case study 51.

1. Problem Statement

Track the number of passengers on a bus across several stops by processing boarding and alighting counts. Report passenger count at each stop and detect capacity violations.

2. Objectives

- Update passenger counts at each stop.
- Validate boarding and alighting values.
- Detect when bus capacity is exceeded.

3. Assumptions

- The bus starts empty.
- Bus capacity is 40 passengers.
- Alighting cannot exceed the number currently on the bus.
- Boarding and alighting counts are non-negative.

4. Algorithm

1. Start.

2. Set passengers to zero and capacity to 40.

3. For each stop, input boarding and alighting counts.

4. Validate counts and update passenger total.

5. Display current occupancy.

6. Warn if occupancy exceeds capacity.

7. Stop.

5. Flowchart (Text Form)

START → Set passengers = 0 → Input boarding/alighting → Validate → Update count → Check capacity → Next stop/STOP

6. Pseudocode

```
passengers = 0; capacity = 40\nFOR each stop:\n  READ boarding, alighting\n  IF invalid or alighting > passengers:
PRINT invalid\n  ELSE:\n    passengers = passengers - alighting + boarding\n    PRINT passengers\n    IF
passengers > capacity: PRINT capacity exceeded
```

7. Python Source Code

```
passengers = 0
capacity = 40
stops = int(input("Number of stops: "))

for stop in range(1, stops + 1):
    print("Stop", stop)
    boarding = int(input("Passengers boarding: "))
    alighting = int(input("Passengers alighting: "))

    if boarding < 0 or alighting < 0 or alighting > passengers:
        print("Invalid passenger counts")
        continue

    passengers = passengers - alighting + boarding
    print("Passengers on bus:", passengers)
```

```
if passengers > capacity:
    print("Warning: bus capacity exceeded")
else:
    print("Within capacity")
```

8. Test Cases

Input / Scenario	Expected result
Stop 1: board 20, alight 0; Stop 2: board 10, alight 5	Occupancy 20 then 25; within capacity
Board 45, alight 0	Occupancy 45; capacity exceeded
At empty bus, alight 2	Invalid passenger counts

9. Sample Output

```
Number of stops: 2
Stop 1
Passengers boarding: 20
Passengers alighting: 0
Passengers on bus: 20
Within capacity
Stop 2
Passengers boarding: 10
Passengers alighting: 5
Passengers on bus: 25
Within capacity
```

Screenshot note: Run the program with the sample input above and insert a screenshot of the console output here if your faculty requires an actual image.

10. Conclusion

The program simulates passenger movement and applies validation and capacity checks at each stop.

8. Food Waste Monitoring System

Selected from the supplied list: Case study 53.

1. Problem Statement

For several days, record meals prepared, meals served, and leftover meals. Calculate the waste percentage and identify days where waste exceeds 10%.

2. Objectives

- Calculate leftover meals.
- Calculate food waste percentage.
- Identify days exceeding the acceptable limit.

3. Assumptions

- Meals prepared must be greater than zero.
- Meals served must be between zero and meals prepared.
- Waste percentage = leftovers / prepared × 100.
- The waste limit is 10%.

4. Algorithm

1. Start.

2. Input number of days.

3. For each day, input prepared and served meals.

4. Validate the data.

5. Calculate leftovers and waste percentage.

6. Report whether waste exceeds 10%.

7. Stop.

5. Flowchart (Text Form)

START → Input daily meals → Validate → Calculate leftovers and waste % → Above 10%? → Report → Next day/STOP

6. Pseudocode

```
READ days\nFOR each day:\n  READ prepared, served\n  IF prepared <= 0 OR served < 0 OR served > prepared: PRINT
invalid\n  ELSE:\n    leftover = prepared - served\n    waste_percent = leftover / prepared * 100\n    PRINT
leftover and waste_percent\n    IF waste_percent > 10: PRINT limit exceeded
```

7. Python Source Code

```
days = int(input("Number of days: "))

for day in range(1, days + 1):
    print("Day", day)
    prepared = int(input("Meals prepared: "))
    served = int(input("Meals served: "))

    if prepared <= 0 or served < 0 or served > prepared:
        print("Invalid meal data")
        continue

    leftover = prepared - served
    waste_percent = leftover / prepared * 100
    print("Leftover meals:", leftover)
    print(f"Waste percentage: {waste_percent:.2f}%")
```

```
if waste_percent > 10:
    print("Warning: waste exceeds 10%")
else:
    print("Waste is within limit")
```

8. Test Cases

Input / Scenario	Expected result
Prepared 100, served 95	Leftover 5; waste 5.00%; within limit
Prepared 100, served 80	Leftover 20; waste 20.00%; exceeds limit
Prepared 0, served 0	Invalid meal data

9. Sample Output

```
Number of days: 1
Day 1
Meals prepared: 100
Meals served: 80
Leftover meals: 20
Waste percentage: 20.00%
Warning: waste exceeds 10%
```

Screenshot note: Run the program with the sample input above and insert a screenshot of the console output here if your faculty requires an actual image.

10. Conclusion

The program calculates food waste and identifies days that need improved meal planning.

9. Password Policy Validation Engine

Selected from the supplied list: Case study 29.

1. Problem Statement

Check whether a password meets a simple policy: at least 8 characters, one uppercase letter, one lowercase letter, one digit, and one special character.

2. Objectives

- Validate password length.
- Check character categories using string methods.
- Display every unmet condition.

3. Assumptions

- A special character is any character that is not a letter or digit.
- This is a classroom demonstration, not a complete real-world security checker.
- The password is evaluated exactly as entered.

4. Algorithm

1. Start.

2. Input a password.

3. Check length, uppercase, lowercase, digit, and special character.

4. Print each failed condition.

5. If none fail, print that the password meets the policy.

6. Stop.

5. Flowchart (Text Form)

START → Input password → Check five rules → Any rule failed? → Print failures or valid password → STOP

6. Pseudocode

```
READ password\nSET failures = empty list\nIF length < 8: add length error\nIF no uppercase: add uppercase error\nIF no lowercase: add lowercase error\nIF no digit: add digit error\nIF no special character: add special-character error\nIF failures empty: PRINT valid\nELSE: PRINT failures
```

7. Python Source Code

```
password = input("Enter password: ")
errors = []

if len(password) < 8:
    errors.append("At least 8 characters are required")
if not any(ch.isupper() for ch in password):
    errors.append("Add an uppercase letter")
if not any(ch.islower() for ch in password):
    errors.append("Add a lowercase letter")
if not any(ch.isdigit() for ch in password):
    errors.append("Add a digit")
if not any(not ch.isalnum() for ch in password):
    errors.append("Add a special character")

if errors:
    print("Password does not meet the policy:")
    for error in errors:
        print("-", error)
```

```
else:
    print("Password meets the policy")
```

8. Test Cases

Input / Scenario	Expected result
Hello@123	Password meets the policy
hello123	Add uppercase letter; add special character
Ab@1	At least 8 characters required

9. Sample Output

```
Enter password: hello123
Password does not meet the policy:
- Add an uppercase letter
- Add a special character
```

Screenshot note: Run the program with the sample input above and insert a screenshot of the console output here if your faculty requires an actual image.

10. Conclusion

The program demonstrates string functions, loops, conditions, and reporting of multiple validation errors.

10. Library Borrowing and Fine Reconciliation

Selected from the supplied list: Case study 58.

1. Problem Statement

Store book borrowing records as tuples containing book ID, days borrowed, and allowed borrowing days. Calculate late-return fines and identify overdue books.

2. Objectives

- Represent borrowing records using tuples.
- Calculate overdue days and fines.
- Summarize total fine.

3. Assumptions

- Each record is (book_id, days_borrowed, allowed_days).
- The fine is Rs. 5 per overdue day.
- Days borrowed and allowed days cannot be negative.
- A book is overdue only when days borrowed is greater than allowed days.

4. Algorithm

1. Start.

2. Input the number of borrowing records.

3. For each record, input book ID, days borrowed, and allowed days.

4. Store values as a tuple.

5. Calculate overdue days and fine.

6. Display each record and total fine.

7. Stop.

5. Flowchart (Text Form)

START → Input borrowing record → Store tuple → Calculate overdue days → Fine = days × Rs. 5 → Display → Next record/STOP

6. Pseudocode

```
total_fine = 0\nREAD n\nFOR each record:\n  READ book_id, days, allowed\n  record = (book_id, days, allowed)\n
overdue = max(0, days - allowed)\n  fine = overdue * 5\n  total_fine += fine\n  PRINT record, overdue, fine\nPRINT
total_fine
```

7. Python Source Code

```
records = []
n = int(input("Number of books: "))
total_fine = 0

for i in range(n):
    book_id = input("Book ID: ")
    days = int(input("Days borrowed: "))
    allowed = int(input("Allowed days: "))

    if days < 0 or allowed < 0:
        print("Invalid day count")
        continue

    records.append((book_id, days, allowed))
```

```
for record in records:
    book_id, days, allowed = record
    overdue = max(0, days - allowed)
    fine = overdue * 5
    total_fine += fine
    print("Book:", book_id, "| Overdue days:", overdue,
          "| Fine: Rs.", fine)

print("Total fine: Rs.", total_fine)
```

8. Test Cases

Input / Scenario	Expected result
B01, borrowed 10 days, allowed 7	Overdue 3 days; fine Rs. 15
B02, borrowed 5 days, allowed 7	Overdue 0 days; fine Rs. 0
B03, borrowed 8 days, allowed 8	Overdue 0 days; fine Rs. 0

9. Sample Output

```
Number of books: 2
Book ID: B01
Days borrowed: 10
Allowed days: 7
Book ID: B02
Days borrowed: 5
Allowed days: 7
Book: B01 | Overdue days: 3 | Fine: Rs. 15
Book: B02 | Overdue days: 0 | Fine: Rs. 0
Total fine: Rs. 15
```

Screenshot note: Run the program with the sample input above and insert a screenshot of the console output here if your faculty requires an actual image.

10. Conclusion

The program uses tuples and arithmetic to identify overdue books and calculate library fines.

Final Submission Checklist

- Review the selected case studies with your faculty.
- Run each Python program and verify its output.
- Capture actual output screenshots from your Python editor or terminal.
- Redraw the text flowcharts using standard flowchart symbols if required.
- Add student name, roll number, class, and faculty details.
- Upload the completed materials to the GitHub repository named PythonActivity-Based-Learning.
- Copy and submit the repository link according to your faculty's instructions.